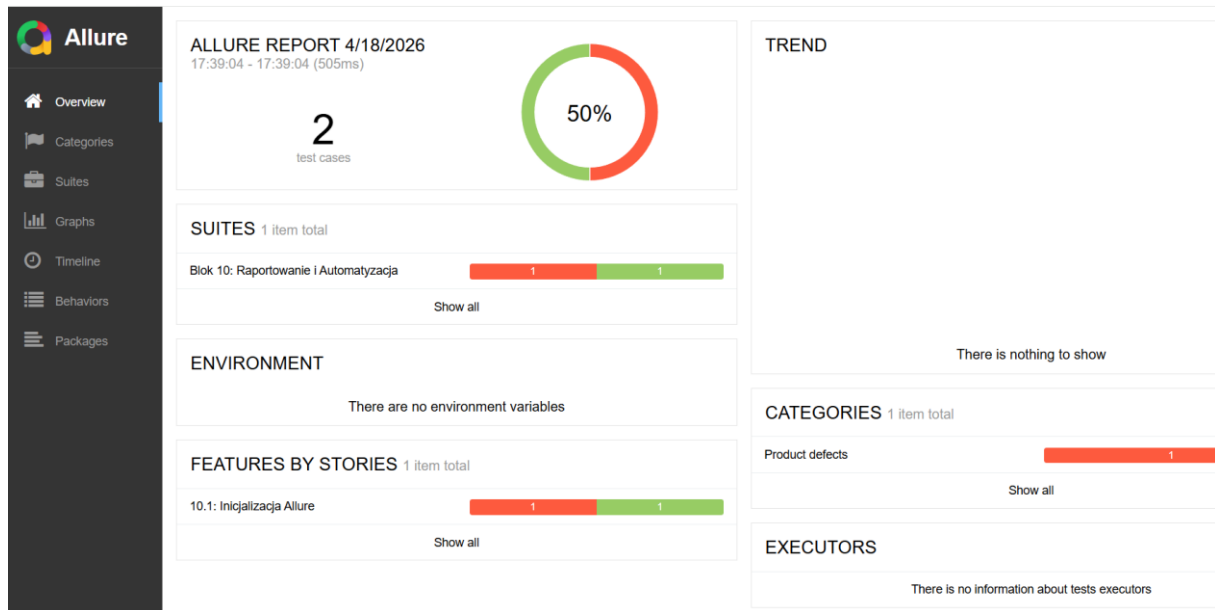


Zadanie 10: Artefakt nr 10 – Raportowanie i Automatyzacja (CI/CD)

10.1. Pytest & Allure Init



10.2. Allure Steps & Decorators

```
PS C:\Users\kamil\Desktop\TM-96407\Artefakt10> python -m pyte
===== test session start =====
platform win32 -- Python 3.14.3, pytest-9.0.3, pluggy-1.6.0
rootdir: C:\Users\kamil\Desktop\TM-96407\Artefakt10
plugins: allure-pytest-2.15.3
collected 2 items

test_102_meta_reporting.py ..
```

10.3. Failure Screenshots

test_103_attachments.TestVisualEvidence#test_screenshot_on_fail

Failed Test ze zrzutem ekranu (Symulacja)

Overview History Retries

Failed: Test padł, ale mamy dowody! Log: ElementNotVisibleException

Categories: Product defects

Severity: normal

Duration: ⌚ 3ms

Execution

▼ Test body

✓ Krok 1: Próba kliknięcia w przycisk 'Zapisz' 2 attachments

2ms

▼  Screenshot_Error_01

 29 B 

Tutaj byłby obraz z telefonu

▼  API_Response

 50 B 

JSON Error Data: {'code': 404, 'msg': 'Not Found'}

Failed: Test padł, ale mamy dowody! Log: ElementNotVisibleException

Failed: Test padł, ale mamy dowody! Log: ElementNotVisibleException

10.4. Clean-up Pipeline

 KROK 3: Dokumentowanie przebiegu Pipeline...

 KROK 4: Generowanie raportu Allure...

Report successfully generated to allure-report

🔧 KROK 5: Zamykanie i usuwanie kontenerów...

[+] down 2/2

✓ Container appium-server Removed

✓ Network artefakt09_mobile-testing Removed

🏆 PIPELINE ZAKOŃCZONY POPRAWNIE!

📄 Raport wygenerowany: C:\Users\kamil\Desktop\TM-96407\Artefakt10\allure-report\index.html

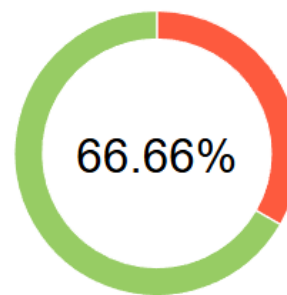
💡 Użyj 'allure serve allure-results' aby go wyświetlić.

ALLURE REPORT 4/18/2026

18:01:59 - 18:01:59 (702ms)

6

test cases



SUITES 3 items total

Blok 10: Raportowanie i Automatyzacja

1

1

test_103_attachments

1

1

test_102_meta_reporting

2

Show all

ENVIRONMENT

Pipeline.Status

SUCCESS

Execution.Time

2026-04-18 18:02:00

Execution.Type

Automated Master Script

Infrastructure

Docker (Appium Server)

Platform

win32

Show all

10.5. Portfolio Build

Mobile Automation & Cloud-Ready Testing Suite

Prowadzący: mgr Mariusz Dworniczak

Student: Kamil Synowiec

Numer Albumu: 96407

Architektura Projektu

Ten projekt to moje kompletne środowisko testowe. Zamiast instalować dziesiątki programów bezpośrednio na komputerze i zaśmiecać system, postawiłem na konteneryzację (Docker) i podejście Cloud-Ready. Dzięki temu cały zestaw narzędzi do testów mobilnych mogę uruchomić na dowolnym komputerze w kilka minut.

Mój Tech Stack:

Język: Python (logika testów i skrypty automatyzujące)

Automatyzacja UI: Appium (mózg operacji na interfejsie telefonu)

Infrastruktura: Docker (izolacja narzędzi od systemu)

Raportowanie: Allure (żeby błędy ładnie wyglądały na wykresach)

Analiza: MobSF & ADB (zagłądanie pod maskę aplikacji APK)

PRZEBIEG LABORATORIUM (Co zrobiłem?)

◆ BLOK 1: Tooling & Environment

Co zrobiono: Skonfigurowałem obrazy Dockera dla Appium i MobSF.

Wniosek: Używamy Dockera, bo dzięki temu nie musimy się martwić o "u mnie nie działa". Każdy ma dokładnie taką samą wersję narzędzi, a po skończonej pracy możemy jednym ruchem wszystko wyłączyć, nie zostawiając śmieci w systemie.

◆ BLOK 2: Debugowanie i Analiza Statyczna (MobSF)

Co zrobiono: Przeskanowałem pliki APK w poszukiwaniu luk bezpieczeństwa.

Wniosek: Analiza statyczna pozwala testerowi dowiedzieć się, do czego

aplikacja chce mieć dostęp (np. mikrofon, kontakty) i czy programista nie zostawił w kodzie jakichś niebezpiecznych "furtek" (jak tryb debugowania), zanim w ogóle klikniemy pierwszy przycisk.

- ♦ BLOK 3-4: Fundamenty Skryptowania (Python for QA)

Co zrobiono: Nauczyłem się pisać skrypty obsługujące listy, słowniki i funkcje, które automatyzują powtarzalne czynności.

Wniosek: Python w testach służy do tego, żeby nie klikać wszystkiego ręcznie. Używałem go do tworzenia warunków (jeśli test padnie -> zrób screena) i pętli.

- ♦ BLOK 5-7: Hybrydowe Testowanie API

Co zrobiono: Testowałem połączenie aplikacji z serwerem za pomocą biblioteki requests. Sprawdzałem, czy serwer zwraca poprawne dane w formacie JSON.

- ♦ BLOK 8: Appium UI Automation

Co zrobiono: Uczyłem się znajdować elementy na ekranie telefonu za pomocą ID oraz XPath. Symulowałem kliknięcia, wpisywanie tekstu i sprawdzanie, czy po kliknięciu wyświetlił się odpowiedni komunikat.

- ♦ BLOK 9: Konteneryzacja Serwera (Docker Compose)

Co zrobiono: Spiąłem wszystko w pliku docker-compose.yml, dzięki czemu serwer Appium rusza razem z całym zapleczem jedną krótką komendą.

- ♦ BLOK 10: MASTER PIPELINE (Finał) 🏆

To "mózg" mojego projektu. Napisałem skrypt pipeline.py, który robi wszystko za mnie:

Odpala Dockera.

Robi testy API (czy serwer żyje).

Robi testy UI (czy aplikacja działa).

Generuje raport Allure ze screenami błędów.

Sprząta po sobie.

Raportowanie Wyników (Allure)

Dzięki Allure nie muszę czytać nudnych logów w konsoli. Widzę:

Dokładne kroki, które wykonał automat.

Zrzuty ekranu z momentu, w którym test się wywalił.

Informacje o tym, na jakim systemie i wersji Pythona ruszyły testy.

Jak to odpalić?

Bash

1. Wejdź do folderu

```
cd Artefakt10
```

2. Uruchom magię

```
python3 pipeline.py
```

3. Zobacz raport w przeglądarce

```
allure serve allure-results
```